

OPEN TECH NOTES

loạt tài liệu kỹ thuật miễn phí cho cộng đồng

RETRIEVAL-AUGMENTED GENERATION

Cho mô hình "tra cứu" trước khi trả lời

Kiến trúc, kỹ thuật truy xuất, và giới hạn thực tế của RAG

Phần 3 trong loạt bài Kỹ thuật AI Engineering

Biên soạn cho cộng đồng IT Việt Nam · Tháng 8, 2026

Mục lục

Mở đầu	3
1. RAG là gì và vì sao cần nó.....	3
2. Kiến trúc RAG cơ bản.....	4
3. Kỹ thuật truy xuất nâng cao	5
3.1. Hybrid search kết hợp ngữ nghĩa và từ khoá	5
3.2. Rerank xếp hạng lại trước khi đưa vào ngữ cảnh.....	5
3.3. Chiến lược chia nhỏ (chunking).....	6
4. Đo lường chất lượng RAG.....	6
5. Rủi ro & giới hạn thực tế	6
5.1. Thất bại trầm lắng của bước truy xuất.....	6
5.2. RAG so với cửa sổ ngữ cảnh lớn và fine-tuning.....	7
6. Checklist nhanh cho đội ngũ.....	7
Đọc thêm	7
Tài liệu tham khảo	8

Mở đầu

Phần 1 của loạt bài nói về foundation model, phần 2 nói về cách giao tiếp với nó qua prompt. Nhưng dù prompt có khéo đến đâu, mô hình vẫn chỉ biết những gì nó đã học trong quá trình huấn luyện dừng lại ở một mốc thời gian cố định (knowledge cutoff), và hoàn toàn không biết gì về dữ liệu riêng của tổ chức bạn: tài liệu nội bộ, cơ sở dữ liệu sản phẩm, chính sách công ty, hay các sự kiện xảy ra sau khi mô hình được huấn luyện.

Retrieval-Augmented Generation (RAG) tạm dịch "sinh văn bản có tăng cường truy xuất" là kỹ thuật giải quyết vấn đề đó: trước khi mô hình trả lời, hệ thống chủ động tìm (truy xuất) những đoạn thông tin liên quan nhất từ một kho dữ liệu bên ngoài, rồi đưa các đoạn đó vào prompt làm ngữ cảnh. Mô hình khi đó không còn phải "nhớ lại" từ trọng số đã huấn luyện, mà được "cho đọc tài liệu" ngay tại thời điểm trả lời giống một người được phép mở sách tra cứu trong lúc thi, thay vì phải thuộc lòng mọi thứ từ trước.

Tài liệu này trình bày RAG là gì và vì sao nó trở thành một trong những kỹ thuật nền tảng của AI Engineering, kiến trúc cơ bản của một hệ thống RAG, các kỹ thuật truy xuất nâng cao thường gặp trong thực tế production, và những giới hạn/rủi ro cần lưu ý. Nội dung là bản tổng hợp gốc dựa trên kiến thức chung của ngành và các nguồn công khai, có trích dẫn ở cuối bài.

1. RAG là gì và vì sao cần nó

Ý tưởng truy xuất thông tin ngoài rồi đưa vào ngữ cảnh trước khi sinh văn bản được trình bày một cách hệ thống trong nghiên cứu "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" (Lewis và cộng sự, NeurIPS 2020) công trình thường được xem là điểm khởi đầu của thuật ngữ RAG trong cộng đồng học thuật. Kể từ đó, RAG đã trở thành một trong những kỹ thuật được áp dụng rộng rãi nhất khi đưa foundation model vào sản phẩm thực tế, vì nó giải quyết trực tiếp ba giới hạn cố hữu của mô hình đã huấn luyện sẵn:

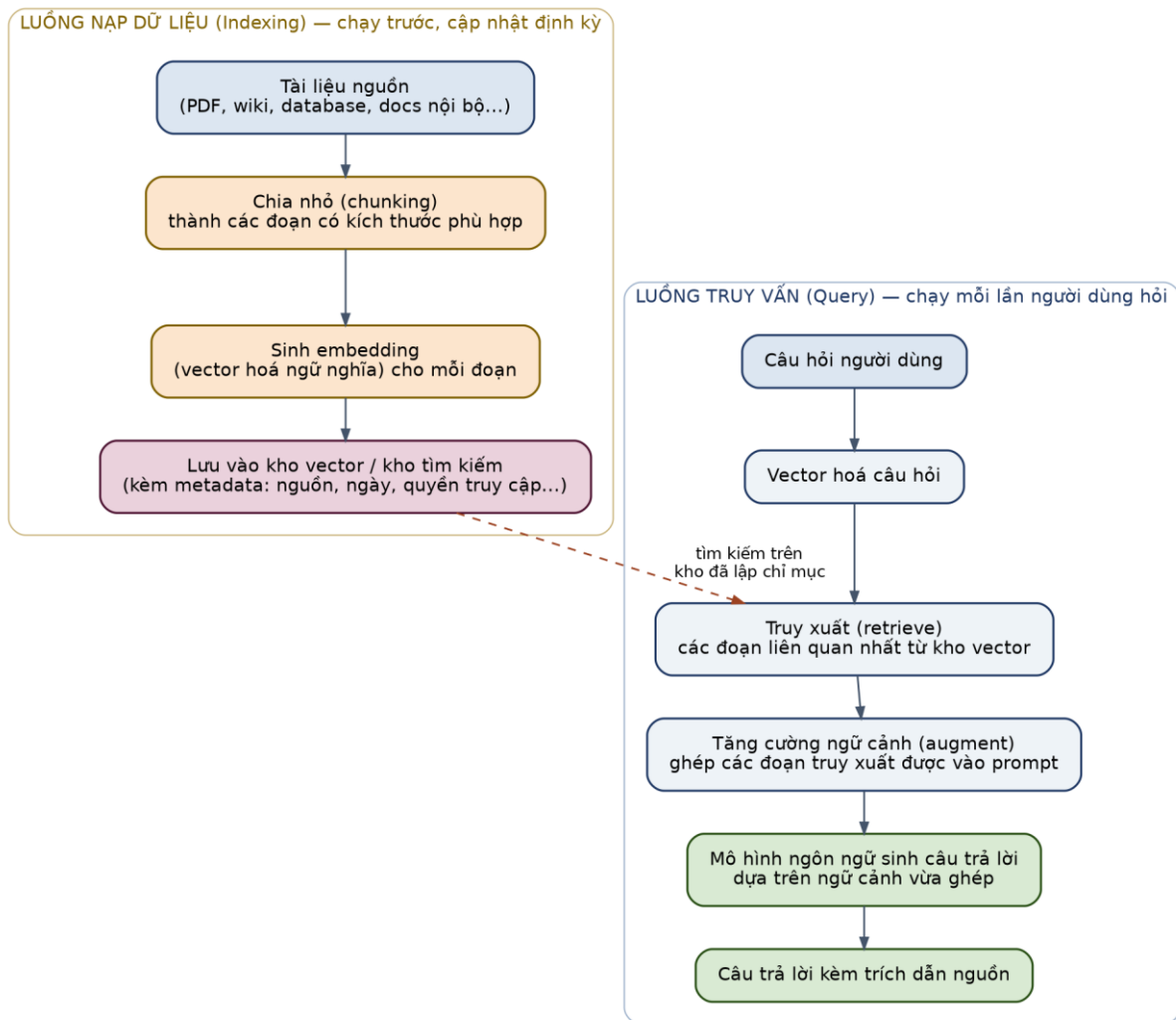
- Kiến thức bị đóng băng tại thời điểm huấn luyện: mô hình không tự biết về những sự kiện, dữ liệu, hay thay đổi xảy ra sau ngày huấn luyện, trừ khi được cung cấp trong ngữ cảnh.
- Không biết dữ liệu riêng tư/nội bộ: tài liệu công ty, hợp đồng khách hàng, cơ sở dữ liệu sản phẩm... không nằm trong dữ liệu huấn luyện công khai của bất kỳ mô hình nào.
- Xu hướng "ảo giác" khi thiếu căn cứ: khi bị hỏi về điều nó không biết, mô hình có xu hướng sinh ra câu trả lời nghe hợp lý nhưng sai sự thật, thay vì thừa nhận không biết cung cấp ngữ cảnh cụ thể để mô hình dựa vào giúp giảm đáng kể rủi ro này.

RAG khác với hai hướng tiếp cận còn lại đã nhắc ở phần 2 của loạt bài. So với việc chỉ đơn thuần nhét toàn bộ tài liệu vào cửa sổ ngữ cảnh (long-context prompting), RAG chọn lọc chỉ những đoạn liên quan nhất, giúp tiết kiệm chi phí/độ trễ và tránh hiện tượng mô hình "lạc" giữa quá nhiều thông tin không liên quan. So với fine-tuning, RAG không thay đổi trọng số mô hình

kho dữ liệu có thể cập nhật gần như tức thời (thêm/xoá tài liệu) mà không cần huấn luyện lại, và câu trả lời có thể kèm trích dẫn nguồn cụ thể, giúp kiểm chứng và tăng độ tin cậy.

2. Kiến trúc RAG cơ bản

Một hệ thống RAG gồm hai luồng xử lý tách biệt, chạy ở những thời điểm khác nhau:



Hình 1. Kiến trúc RAG cơ bản: luồng nạp dữ liệu (indexing) chạy trước và cập nhật định kỳ, luồng truy vấn (query) chạy mỗi lần người dùng đặt câu hỏi.

Luồng nạp dữ liệu (indexing) chuẩn bị kho tri thức trước: tài liệu nguồn được chia nhỏ thành các đoạn (chunking), mỗi đoạn được chuyển thành một vector số (embedding) đại diện cho ý nghĩa ngữ nghĩa của nó, rồi lưu vào một kho vector (vector store) kèm metadata như nguồn gốc, ngày cập nhật, hay quyền truy cập. Bước này thường chạy theo lô (batch), định kỳ hoặc mỗi khi có tài liệu mới.

Luồng truy vấn (query) chạy mỗi khi có câu hỏi: câu hỏi được vector hoá bằng cùng mô hình embedding, dùng để truy xuất các đoạn có vector gần nhất trong kho (thường đo bằng độ tương

đồng cosine), các đoạn truy xuất được ghép vào prompt làm ngữ cảnh, và mô hình ngôn ngữ sinh câu trả lời cuối cùng dựa trên ngữ cảnh đó. Lý tưởng nhất là kèm trích dẫn cho biết câu trả lời dựa trên đoạn tài liệu nào.

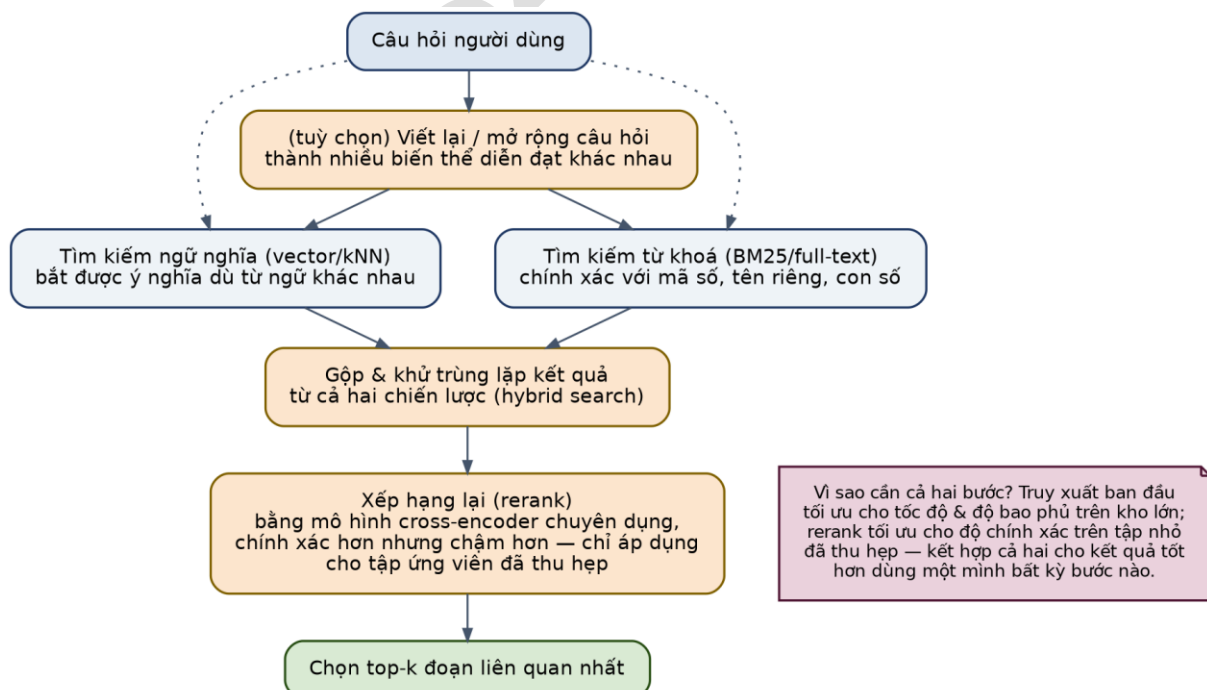
3. Kỹ thuật truy xuất nâng cao

3.1. Hybrid search kết hợp ngữ nghĩa và từ khoá

Tìm kiếm bằng vector (semantic search) rất mạnh khi câu hỏi và tài liệu diễn đạt khác từ ngữ nhưng cùng ý nghĩa, nhưng lại yếu ở những trường hợp cần khớp chính xác: mã sản phẩm, tên riêng, số phiên bản, thuật ngữ hiếm. Tìm kiếm từ khoá truyền thống (thường dùng thuật toán BM25, một cải tiến của mô hình xác suất TF-IDF) lại mạnh đúng ở điểm yếu đó. Hybrid search chạy song song cả hai chiến lược, rồi gộp và khử trùng lặp kết quả. Cách tiếp cận này thường cho độ chính xác truy xuất cao hơn đáng kể so với dùng riêng lẻ một trong hai, và là lựa chọn mặc định hợp lý cho phần lớn hệ thống RAG trong thực tế.

3.2. Rerank xếp hạng lại trước khi đưa vào ngữ cảnh

Bước truy xuất ban đầu (vector hoặc hybrid) thường được tối ưu cho tốc độ, quét qua kho dữ liệu lớn để lấy ra một tập ứng viên rộng (ví dụ top 50–100 đoạn). Bước rerank sau đó dùng một mô hình chuyên biệt hơn — thường là cross-encoder, chấm điểm mức độ liên quan giữa câu hỏi và từng đoạn ứng viên một cách chính xác hơn nhiều so với so sánh vector đơn thuần, dù chậm hơn — để xếp hạng lại và chỉ giữ lại một tập nhỏ (ví dụ top 5) đưa vào ngữ cảnh cuối cùng.



Hình 2. Hybrid search kết hợp tìm kiếm ngữ nghĩa và từ khoá, sau đó rerank bằng cross-encoder trước khi chọn ra tập kết quả cuối cùng.

Nguyên tắc chung: bước truy xuất đầu tối ưu cho việc không bỏ sót (recall) trên toàn bộ kho dữ liệu, còn rerank tối ưu cho độ chính xác (precision) trên một tập nhỏ đã thu hẹp kết hợp cả hai giai đoạn thường hiệu quả hơn hẳn so với chỉ dùng một bước duy nhất.

3.3. Chiến lược chia nhỏ (chunking)

Cách chia tài liệu thành các đoạn (chunk) ảnh hưởng trực tiếp đến chất lượng truy xuất, dù thường bị xem nhẹ so với việc chọn mô hình embedding. Đoạn quá nhỏ dễ mất ngữ cảnh xung quanh (một câu trích riêng lẻ có thể tối nghĩa nếu tách khỏi đoạn văn chứa nó); đoạn quá lớn làm loãng vector đại diện, khiến việc tìm đúng phần liên quan trở nên khó hơn và tốn nhiều token ngữ cảnh hơn mức cần thiết. Vài chiến lược phổ biến: chia theo kích thước cố định kèm phần chồng lấn (overlap) giữa các đoạn liên kề để không cắt đứt ý giữa chừng; chia theo cấu trúc tài liệu (đoạn văn, mục, tiêu đề) thay vì cắt cơ học theo số ký tự; và chia theo ngữ nghĩa (semantic chunking), dùng chính mô hình embedding để phát hiện điểm chuyển ý và cắt tại đó. Một kỹ thuật bổ trợ đáng chú ý là thêm ngữ cảnh ngắn gọn về vị trí/nội dung tổng thể của tài liệu vào đầu mỗi đoạn trước khi tạo embedding giúp đoạn nhỏ vẫn giữ được ý nghĩa khi được truy xuất riêng lẻ, một kỹ thuật đã được Anthropic mô tả công khai dưới tên gọi "contextual retrieval".

4. Đo lường chất lượng RAG

Một hệ thống RAG có hai lớp cần đo riêng biệt, vì lỗi có thể nằm ở bất kỳ lớp nào. Chất lượng truy xuất đo việc kho dữ liệu có tìm ra đúng đoạn liên quan hay không độc lập với việc mô hình trả lời như thế nào thường dùng các chỉ số như precision@k (trong k đoạn lấy ra, bao nhiêu đoạn thực sự liên quan) và recall@k (trong tất cả đoạn liên quan tồn tại, tìm được bao nhiêu). Chất lượng câu trả lời cuối cùng đo việc mô hình có dùng đúng và đủ ngữ cảnh được cấp hay không có thể vẫn sai dù truy xuất đúng, nếu mô hình bỏ sót thông tin quan trọng trong ngữ cảnh hoặc trộn lẫn với kiến thức nội tại sai lệch.

Đây chỉ là phần giới thiệu ngắn gọn cách xây dựng bộ đánh giá (evaluation set), chọn chỉ số phù hợp theo bài toán, và quy trình đánh giá tự động ở quy mô lớn sẽ được trình bày đầy đủ trong tài liệu riêng "Đánh giá mô hình" của loạt bài này.

5. Rủi ro & giới hạn thực tế

5.1. Thất bại thầm lặng của bước truy xuất

Khác với lỗi hệ thống thông thường (crash, timeout) vốn dễ phát hiện, một hệ thống RAG có thể vận hành trơn tru về mặt kỹ thuật trong khi âm thầm trả lời sai vì bước truy xuất lấy nhầm đoạn không liên quan, hoặc bỏ sót đúng đoạn cần thiết, mà không có tín hiệu lỗi nào để nhận biết. Mô hình khi đó thường vẫn cố sinh ra một câu trả lời có vẻ hợp lý dựa trên ngữ cảnh sai,

thay vì nhận ra ngữ cảnh không đủ để trả lời một dạng ảo giác đặc biệt khó phát hiện vì câu trả lời trông rất tự tin. Giảm thiểu rủi ro này cần: chỉ dẫn rõ ràng cho mô hình được phép nói "không tìm thấy thông tin liên quan" khi ngữ cảnh không đủ, theo dõi (log) các đoạn được truy xuất cho từng câu hỏi để có thể gỡ lỗi ngược, và một bộ đánh giá độc lập cho riêng bước truy xuất như đã nói ở phần 4.

5.2. RAG so với cửa sổ ngữ cảnh lớn và fine-tuning

Khung quyết định đã giới thiệu ở phần 2 của loạt bài vẫn áp dụng ở đây: RAG là lựa chọn phù hợp khi vấn đề là thiếu kiến thức/dữ liệu cụ thể, đặc biệt là dữ liệu hay thay đổi hoặc cần trích dẫn nguồn không phải khi vấn đề là hành vi/phong cách cần nhất quán ở quy mô lớn (khi đó fine-tuning phù hợp hơn), và cũng không thay thế được prompt engineering tốt ở lớp chỉ dẫn cơ bản. Một câu hỏi thực tế hay gặp là: với các mô hình có cửa sổ ngữ cảnh ngày càng lớn, liệu có thể bỏ qua RAG và nhét thẳng toàn bộ tài liệu vào prompt? Với kho dữ liệu nhỏ, cách này khả thi và đơn giản hơn; nhưng khi kho dữ liệu lớn, RAG vẫn ưu việt hơn về chi phí (chỉ trả token cho phần ngữ cảnh thực sự cần), độ trễ, và độ chính xác mô hình có xu hướng xử lý kém hơn khi phải "lọc" thông tin liên quan giữa một lượng lớn nội dung không liên quan trong ngữ cảnh dài.

6. Checklist nhanh cho đội ngũ

- Chiến lược chunking có phù hợp với cấu trúc tài liệu nguồn, hay chỉ cắt cơ học theo số ký tự?
- Đã thử hybrid search (ngữ nghĩa + từ khoá) thay vì chỉ dùng riêng vector search chưa?
- Có bước rerank trước khi đưa ngữ cảnh vào prompt cuối cùng, đặc biệt khi tập ứng viên ban đầu lớn?
- Mô hình có được phép và được khuyến khích trả lời "không tìm thấy thông tin liên quan" khi ngữ cảnh không đủ?
- Có đo lường chất lượng truy xuất (precision/recall@k) tách biệt với chất lượng câu trả lời cuối cùng?
- Có log lại các đoạn được truy xuất cho từng câu hỏi để gỡ lỗi khi câu trả lời sai?
- Kho vector có được cập nhật/xoá tương ứng khi tài liệu nguồn thay đổi hoặc bị thu hồi quyền truy cập?

Đọc thêm

Tài liệu này tập trung vào các nguyên tắc kiến trúc và kỹ thuật nền tảng của RAG. Nếu bạn muốn đào sâu hơn với một góc nhìn hệ thống, đầy đủ ví dụ mã nguồn và case study thực chiến

về xây dựng hệ thống truy xuất trong bối cảnh ứng dụng production trên nền foundation model, cuốn sách sau là một nguồn tham khảo chuyên sâu đáng đọc:

"AI Engineering: Building Applications with Foundation Models" Chip Huyen, O'Reilly Media, 2025.

Bản tiếng Việt phát hành bởi Times, có bán tại các nhà sách và sàn thương mại điện tử.

Tài liệu bạn đang đọc là nội dung gốc, biên soạn độc lập dựa trên kiến thức chung của ngành và các nguồn công khai liệt kê bên dưới không phải bản dịch hay tóm tắt của cuốn sách trên.

Tài liệu tham khảo

- Lewis, P. et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS, 2020 (arXiv:2005.11401).
- Robertson, S., Zaragoza, H. "The Probabilistic Relevance Framework: BM25 and Beyond." Foundations and Trends in Information Retrieval, 2009.
- Nogueira, R., Cho, K. "Passage Re-ranking with BERT." arXiv:1901.04085, 2019.
- Anthropic. "Introducing Contextual Retrieval." anthropic.com/news, 2024.
- Pinecone. "Chunking Strategies for LLM Applications." pinecone.io/learn, 2026.
- LangChain. "Retrieval." python.langchain.com/docs, 2026.